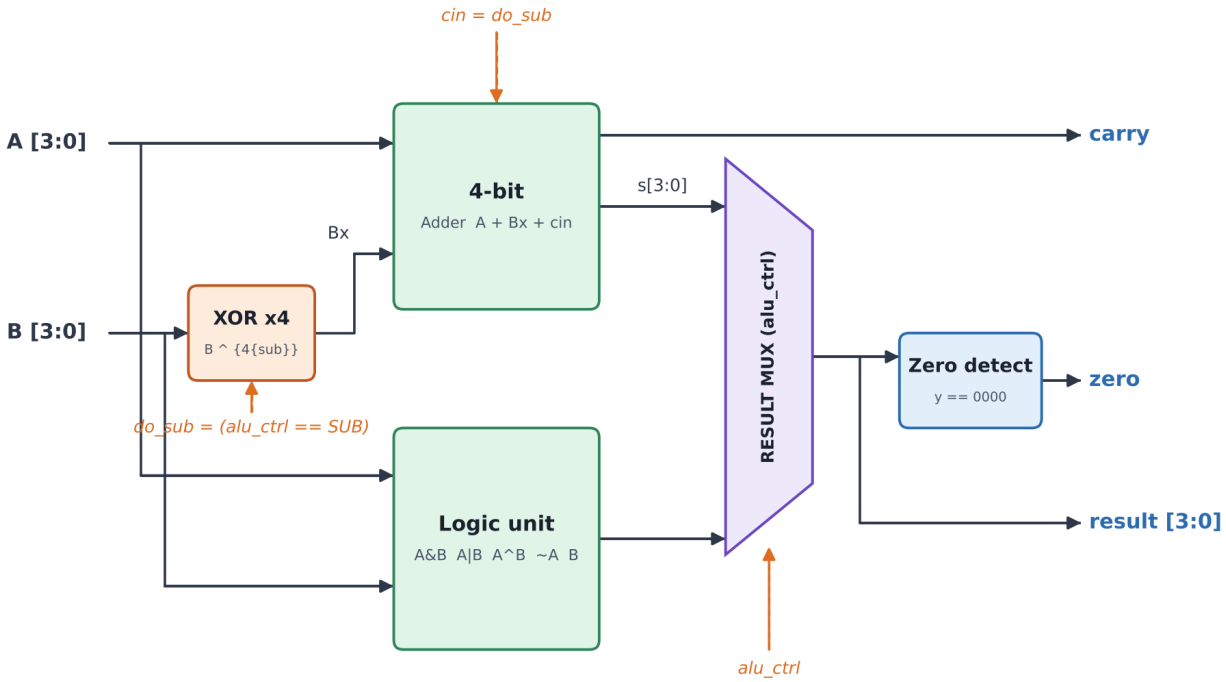
Code walkthrough

alu.v

ALU (Arithmetic Logic Unit)

What it does	Does all calculations
Where it works	EX stage
Size	143 lines, explained in 11 small parts
Modules in this file	adder_subtractor, logic_unit, zero_detector, alu
Calls (uses)	only modules inside this same file: adder_subtractor, logic_unit, zero_detector
Called by	cpu_top.v - instance ALU tb_alu.v - instance DUT

The ALU takes two 4-bit numbers and an operation code and produces a 4-bit result plus the Carry and Zero flags. The file is split into four small modules: three helpers and the main ALU that joins them.



Inside the ALU: the helper modules and the result multiplexer.

Map of the file

Part	Lines	What this part is
1	1-29	Header comment
2	32-42	Module 1: ports and the 5-bit sum
3	44-51	Module 1: ADD
4	52-61	Module 1: SUB
5	64-83	Module 2: logic_unit
6	86-94	Module 3: zero_detector
7	97-107	Module 4: ports of the main ALU
8	108-112	Module 4: internal wires
9	114-124	Module 4: using the helpers
10	126-136	Module 4: choosing the answer
11	138-143	Module 4: zero flag

How to read this document: each part below shows a few lines of the real file, with their line numbers, followed by a plain-language explanation. Read the parts in order to understand the whole file without scrolling through it in one go.

Part 1 · Header comment (lines 1-29)

alu.v lines 1-29

```
 1  // ======================================================================
 2  // FILE: alu.v
 3  // The ALU (Arithmetic Logic Unit) does all the calculations.
 4  //
 5  // Interface (as in the project plan):
 6  //   a[3:0], b[3:0], alu_ctrl, result[3:0], carry, zero
 7  //
 8  // This file contains 4 small modules:
 9  //   1. adder_subtractor : does ADD and SUB, gives the carry
10  //   2. logic_unit       : does AND, OR, XOR, NOT and PASS (copy b)
11  //   3. zero_detector    : checks if the result is 0000
12  //   4. alu              : the main ALU, it joins the 3 parts above
13  //
14  // ALU control codes (alu_ctrl):
15  //   0 = ADD    1 = SUB    2 = AND    3 = OR
16  //   4 = XOR    5 = NOT    6 = PASS (result = b; used by LDI, MOV,
17  //                                   LOAD and STORE)
18  //
19  // CARRY CONVENTION (same in ALU, flag register and testbenches):
20  //   ADD : carry = 1 when the sum is bigger than 15 (carry-out)
21  //   SUB : carry = 1 when a >= b, i.e. NO BORROW
22  //         carry = 0 when a <  b, i.e. a borrow happened
23  //
24  // CONNECTIONS
25  //   Calls       : only modules inside this same file:
26  //                 adder_subtractor, logic_unit, zero_detector
27  //   Called by   : cpu_top.v - instance ALU
28  //                 tb_alu.v - instance DUT
29  // ======================================================================
```

- Lists the four modules in the file and the ALU control codes (0 = ADD ... 6 = PASS).
- Documents the **carry convention**: for ADD, carry = 1 means the sum did not fit; for SUB, carry = 1 means no borrow (a >= b). The same rule is used everywhere in the project.
- The **CONNECTIONS** lines at the end of the header list which other files this file uses (Calls) and which files use it (Called by).

Part 2 · Module 1: ports and the 5-bit sum (lines 32-42)

alu.v lines 32-42

```
32  // ------------------------------------------------------------------
33  // MODULE 1: adder_subtractor
34  // ------------------------------------------------------------------
35  module adder_subtractor (
36      input      [3:0] a,
37      input      [3:0] b,
38      input            do_sub,     // 0 = add, 1 = subtract
39      output reg [3:0] result,
40      output reg       carry
41  );
42      reg [4:0] full_sum;          // 5 bits, so we can see the carry bit
```

- Inputs a, b and do_sub (0 = add, 1 = subtract). Outputs result and carry.
- `reg [4:0] full_sum` is 5 bits wide: 4 bits for the answer plus one extra bit to catch the carry.

Part 3 · Module 1: ADD (lines 44-51)

alu.v lines 44-51

```
44      always @(*) begin
45          full_sum = 5'b00000;
46          if (do_sub == 1'b0) begin
47              // ---- ADD ----
48              full_sum = a + b;            // e.g. 15 + 1 = 16 = 1_0000
49              result   = full_sum[3:0];    // lower 4 bits are the answer
50              carry    = full_sum[4];      // 5th bit is the carry
51          end
```

- `full_sum = a + b`. Example: 15 + 1 = 16 = 1_0000.
- `result = full_sum[3:0]` takes the lower 4 bits (0000).
- `carry = full_sum[4]` takes the 5th bit (1): the answer did not fit in 4 bits.
- `full_sum = 0` at the start just gives it a value in every case (avoids a latch).

Part 4 · Module 1: SUB (lines 52-61)

alu.v lines 52-61

```
52          else begin
53              // ---- SUB ----
54              result = a - b;              // e.g. 5 - 3 = 2
55              if (a >= b)
56                  carry = 1'b1;            // no borrow
57              else
58                  carry = 1'b0;            // borrow happened
59          end
60      end
61  endmodule
```

- `result = a - b`. Example: 5 - 3 = 2. (3 - 5 wraps round to 1110.)
- Carry is set by a simple comparison: 1 if a >= b (no borrow), 0 if a < b (borrow).
- The synthesis tool builds this as the usual adder with B inverted and carry-in = 1.

Part 5 · Module 2: logic_unit (lines 64-83)

alu.v lines 64-83

```
64  // ------------------------------------------------------------------
65  // MODULE 2: logic_unit
66  // ------------------------------------------------------------------
67  module logic_unit (
68      input      [3:0] a,
69      input      [3:0] b,
70      input      [2:0] alu_ctrl,
71      output reg [3:0] result
72  );
73      always @(*) begin
74          case (alu_ctrl)
75              3'd2:    result = a & b;     // AND
76              3'd3:    result = a | b;     // OR
77              3'd4:    result = a ^ b;     // XOR
78              3'd5:    result = ~a;        // NOT (only uses a)
79              3'd6:    result = b;         // PASS: copy b
80              default: result = 4'b0000;
81          endcase
82      end
83  endmodule
```

- One case on `alu_ctrl`: 2 = AND, 3 = OR, 4 = XOR, 5 = NOT (uses only a), 6 = PASS (copies b).
- PASS is used by LDI, MOV, LOAD and STORE to send a value straight through the ALU.
- `default` gives 0000 for any other code, so the output is always defined.

Part 6 · Module 3: zero_detector (lines 86-94)

alu.v lines 86-94

```
86  // ------------------------------------------------------------------
87  // MODULE 3: zero_detector
88  // ------------------------------------------------------------------
89  module zero_detector (
90      input  [3:0] value,
91      output       zero
92  );
93      assign zero = (value == 4'b0000) ? 1'b1 : 1'b0;
94  endmodule
```

One line: zero = 1 if the value is 0000, otherwise 0.

Part 7 · Module 4: ports of the main ALU (lines 97-107)

alu.v lines 97-107

```
 97  // ------------------------------------------------------------------
 98  // MODULE 4: alu (main ALU)
 99  // ------------------------------------------------------------------
100  module alu (
101      input      [3:0] a,          // operand A (register Rd)
102      input      [3:0] b,          // operand B (register Rs or immediate)
103      input      [2:0] alu_ctrl,   // which operation to do
104      output reg [3:0] result,     // final answer
105      output reg       carry,      // carry / no-borrow
106      output           zero        // 1 when result = 0000
107  );
```

a is the value of Rd, b is Rs or the immediate (chosen by the operand MUX), alu_ctrl comes from the control unit. Outputs: result, carry, zero.

Part 8 · Module 4: internal wires (lines 108-112)

alu.v lines 108-112

```
108     wire [3:0] arith_result;     // answer from adder_subtractor
109     wire       arith_carry;      // carry  from adder_subtractor
110     wire [3:0] logic_result;     // answer from logic_unit
111
112     wire do_sub = (alu_ctrl == 3'd1);    // alu_ctrl 1 means SUB
```

- Three wires carry the answers of the helper modules into the main ALU.
- do_sub is 1 only when alu_ctrl is 1 (SUB); it tells the adder to subtract.

Part 9 · Module 4: using the helpers (lines 114-124)

alu.v lines 114-124

```
114     // CALLS: module adder_subtractor (MODULE 1 in this same file, alu.v)
115     adder_subtractor ADDSUB (
116         .a(a), .b(b), .do_sub(do_sub),
117         .result(arith_result), .carry(arith_carry)
118     );
119
120     // CALLS: module logic_unit (MODULE 2 in this same file, alu.v)
121     logic_unit LOGIC (
122         .a(a), .b(b), .alu_ctrl(alu_ctrl),
123         .result(logic_result)
124     );
```

- These are **instances**: real copies of modules 1 and 2 placed inside the ALU. `.a(a)` means "connect the helper's port a to my signal a".
- Both helpers work all the time in parallel; the next chunk chooses which answer to use.

→ This part **calls module** `adder_subtractor`, which is MODULE 1 in this same file (see the earlier parts of this document).

→ This part **calls module** `logic_unit`, which is MODULE 2 in this same file (see the earlier parts of this document).

Part 10 · Module 4: choosing the answer (lines 126-136)

alu.v lines 126-136

```
126     // choose the final answer
127     always @(*) begin
128         if (alu_ctrl == 3'd0 || alu_ctrl == 3'd1) begin
129             result = arith_result;       // ADD or SUB
130             carry  = arith_carry;
131         end
132         else begin
133             result = logic_result;       // AND, OR, XOR, NOT, PASS
134             carry  = 1'b0;               // logic ops have no carry
135         end
136     end
```

- If alu_ctrl is 0 or 1 (ADD or SUB), the answer and carry come from the adder.
- Otherwise the answer comes from the logic unit and carry is 0.
- Both outputs get a value in both branches, so no latch is created.

Part 11 · Module 4: zero flag (lines 138-143)

alu.v lines 138-143

```
138      // CALLS: module zero_detector (MODULE 3 in this same file, alu.v)
139      zero_detector ZERO (
140          .value(result),
141          .zero(zero)
142      );
143  endmodule
```

The final result goes into the zero detector, so Zero is correct for every operation.

→ This part **calls module** `zero_detector`, which is MODULE 3 in this same file (see the earlier parts of this document).

> **In short:** Two helpers compute in parallel; the main ALU picks the right answer and adds the Zero flag.